

Practical 7

AIM: Implementation of making a change problem using dynamic programming

1 ALGORITHM:

Algorithm:-
 Input :- coin denominations, coins().
 number of denominations:- n.
 target amount:- V.
 output:- minimum number of coins amount V.
step 1:-
 Initialize an array dp [0..V] where.
 target amount V.
 set $dp[0] = 0$ (no. of coin needed for amount 0).
 for all the i, set $dp[i] = \infty$ Initially.
 For each amount i to V:-
 for each coins [i].
 if coins [j] $\leq i$ upto.
 $dp[i] = \min(dp[i - coins[j]] + 1)$
 After finding the value $dp[V]$ given minimum.
 if $dp[V] = \infty$, no solution exists.

2 PROGRAM:

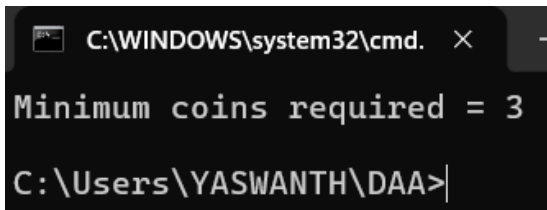
```
#include <iostream>
#include <climits>
using namespace std;
int minCoins(int coins[], int m, int V) {
    int dp[V+1];
    dp[0] = 0;
    for (int i = 1; i <= V; i++)
        dp[i] = INT_MAX;
    for (int i = 1; i <= V; i++) {
        for (int j = 0; j < m; j++) {
            if (coins[j] <= i) {
                int sub_res = dp[i - coins[j]];
                if (sub_res != INT_MAX && sub_res + 1 < dp[i])
                    dp[i] = sub_res + 1;
            }
        }
    }
    return dp[V];
}
int main() {
    int coins[] = {1, 5, 10};
    int m = sizeof(coins)/sizeof(coins[0]);
```

```
int V = 12;

int result = minCoins(coins, m, V);
if (result == INT_MAX)
    cout << "No solution possible" << endl;
else
    cout << "Minimum coins required = " << result << endl;

return 0;
}
```

3 OUTPUT:



```
C:\WINDOWS\system32\cmd. X
Minimum coins required = 3
C:\Users\YASWANTH\DA>
```

4 TIME COMPLEXITY:

Time complexity
Outer loop (amount 1) runs for 0 to V.
→ V times.
Inner loop (runs for 0 to n-1)
m times for each amount.
So, $O(1)$ per iteration.
Total operations: $V \times m \times O(1)$
 $T(V, m) = O(m \cdot V)$

5 Conclusion:

The Making Change problem solved with Dynamic Programming ensures the minimum number of coins is found by building solutions for smaller amounts and combining them optimally.